

Legal Connect

Collecte, entraînement et intégration des données juridiques
dans le chatbot et le module de recommandation d'experts

Document technique - Memoire Master 1 MIAGE
Universite Nangui Abrogoua - Cote d'Ivoire

1. Objectif de ce document

Ce document decrit, de maniere factuelle et verifiable, comment les donnees juridiques ivoiriennes reelles (articles de loi + reformulations) ont ete collectees, puis integrees dans deux composants distincts de la plateforme Legal Connect : le **chatbot juridique** (recherche d'articles reels, approche RAG - Retrieval-Augmented Generation) et le **module de recommandation d'experts** (classifieur TF-IDF + LinearSVC). Les deux integrations reposent sur les *memes* fichiers sources mais les exploitent differemment, pour des raisons methodologiques expliquees plus loin.

2. Collecte des donnees

2.1 Format des fichiers

Les donnees se presentent sous la forme de 6 fichiers CSV ('lots'), chacun couvrant un domaine du droit ivoirien. Chaque ligne comporte 4 colonnes :

- **text** : le contenu (texte de l'article de loi, ou reformulation en question)
- **label** : la categorie juridique (droit_travail, droit_commercial, droit_immobilier, droit_fiscal, creation_entreprise, droit_famille, droit_penal, propriete_intel)
- **source** : la reference exacte du texte de loi (ex. "Code Civil II, art. 605")
- **type** : soit *article* (texte de loi original), soit *query_variant* (reformulation de l'article sous forme de question, associee a la meme reference)

Chaque article de loi possede donc, dans le meme lot, une ligne 'article' (texte source) et une ou plusieurs lignes 'query_variant' (question reformulee). Cette structure a ete determinante pour le choix des methodes d'integration : les lignes 'article' sont utilisees comme **source de verite a citer**, les lignes 'query_variant' comme **exemples d'entrainement** car plus proches du registre d'un utilisateur reel.

2.2 Volumetrie par lot

Fichier	Categorie(s)	Lignes	Articles uniques
dataset_droit_penal_lot1.csv	droit_penal	1 830	915
dataset_lot2_ohada_commercial.csv	droit_commercial, creation_entreprise, droit_fiscal	4 130	993
dataset_lot3_civil.csv	droit_immobilier, droit_commercial	2 306	1 153
dataset_lot5_travail.csv	droit_travail	126	63
dataset_lot6_famille.csv	droit_famille	132	61
dataset_lot7_propint.csv	propriete_intel	68	34
TOTAL	8 categories	9 392	3 219

Les labels correspondent exactement aux 8 categories deja utilisees par le module de recommandation (CATEGORIES_LABELS dans ml_recommandation/classifier.py), ce qui a permis une integration directe sans travail de mapping.

3. Integration dans le chatbot (recherche RAG)

3.1 Principe

Plutot que d'entrainer un classifieur, le chatbot utilise les articles de loi comme une **base de connaissances interrogeable** : face a une question, il cherche l'article le plus proche par similarite de texte (TF-IDF + similarite cosinus) et peut citer son contenu exact avec sa reference legale, plutot que de se contenter d'une reponse generique.

3.2 Pipeline technique

- **chatbot/data/*.csv** : copie des 6 lots (extensible - tout nouveau fichier dataset_*.csv depose ici est pris en compte automatiquement)
- **chatbot/construire_index.py** : lit les CSV, nettoie le texte (minuscule, suppression des accents, retrait des mots-outils), vectorise l'ensemble (article + query_variant) avec un TfidfVectorizer (n-grammes 1-2), puis sauvegarde le vectoriseur et la matrice TF-IDF
- **chatbot/index/*.pkl** : index resultant (vectoriseur, matrice, metadonnees), 8 Mo, 83 458 termes de vocabulaire
- **chatbot/kb.py** : module de recherche - charge l'index une fois par processus, compare une question a l'ensemble du corpus, retourne les articles les plus proches dedupliques par reference (fonctions rechercher(), meilleur_article(), contexte_pour_llm())

3.3 Utilisation dans les deux moteurs du chatbot

Moteur	Role de la recherche RAG	Seuil de confiance
chatbot/moteur.py (regles, sans IA)	Cite l'article reel en complement ou en reponse principale si aucune regle ne s'applique	0.30 (tres selectif)
chatbot/moteur_ia.py (Claude, production)	Injecte les articles pertinents dans le prompt systeme de Claude, qui les cite (ou les ignore s'il ne les trouve pas)	0.15 (Claude filtre la pertinence)

3.4 Limite connue

Le Code Civil (lot3, ~1800 articles de droit napoleonien traduit) domine numeriquement le corpus et son vocabulaire generique (obligations, contrats) peut faire remonter des articles hors-sujet par polysemie - par exemple le mot "caution" designe en Code Civil le *cautionnement* (garant d'une dette), pas le *depot de garantie locatif* attendu par un utilisateur. Le seuil de 0.30 dans moteur.py filtre la plupart des faux positifs, mais le corpus ne couvre pas non plus le bail locatif moderne, la creation de SARL (demarches administratives) ni la TVA/CGI de maniere satisfaisante - un renforcement cible sur ces sujets ameliorerait la precision.

Par ailleurs, le texte de loi brut etant redige en francais juridique (parfois ancien), il n'est pas comprehensible tel quel par un non-juriste. Il est donc toujours presente comme une reference secondaire ("voici le texte exact, un professionnel peut vous en expliquer le sens"), jamais comme une explication suffisante en elle-meme.

4. Entrainement et integration dans la recommandation d'experts

4.1 Pipeline existant avant integration

Le module ml_recommandation/entrainer.py utilisait deja un pipeline TF-IDF (n-grammes 1-3) + LinearSVC, entraine sur un dataset synthetique de 200 phrases redigees a la main (25 par categorie, dans ml_recommandation/data/dataset.json), augmente par repetition de 20 phrases-modeles par categorie

(TEMPLATES), soit 840 exemples au total.

4.2 Choix methodologique : quelles lignes reutiliser ?

Contrairement au chatbot, le classifieur de recommandation reçoit en entree la description libre d'un probleme par un utilisateur ("mon employeur ne me paie pas..."), pas une question sur un article de loi. Deux options ont ete testees par ablation :

- Integrer les lignes **article** (texte de loi brut) : rejete, registre trop eloigne d'une description de probleme reelle, risque de bruit lexical important
- Integrer les lignes **query_variant** (reformulations en question) : retenu, registre plus proche, tout en restant du vocabulaire juridique authentique

Un plafond par categorie a egalement ete necessaire : sans lui, la categorie droit_commercial (2 465 query_variants a elle seule, issus du droit civil general des obligations + OHADA) écraserait numeriquement les categories plus modestes comme droit_famille (66) ou propriete_intel (34), et diluerait le classifieur avec du vocabulaire trop generique.

4.3 Validation par ablation du plafond

Le plafond ideal a ete determine empiriquement, en construisant un jeu de 20 questions realistes et informelles (non presentes dans les donnees d'entrainement, redigees specifiquement pour ce test) et en mesurant le taux de bonne categorisation :

Configuration	Score sur 20 questions test	F1 (validation croisee 5-fold)
Sans donnees reelles (baseline)	17/20 (85%)	0.991 *
+ donnees reelles, plafond 150/categorie	17/20 (85%)	0.878
+ donnees reelles, plafond 100/categorie	18/20 (90%)	0.898
+ donnees reelles, plafond 50/categorie (retenu)	18/20 (90%)	0.928

* Le score de validation croisee de la baseline est artificiellement eleve : il est calcule sur des donnees generees de maniere homogene (memes templates repetes), ce qui surestime la capacite de generalisation reelle du modele - d'ou l'interet du jeu de 20 questions independant pour trancher.

4.4 Bug corrige : desynchronisation entrainement / prediction

Au cours de cette validation, un ecart a ete detecte entre la fonction de nettoyage de texte utilisee a l'entrainement (ml_recommandation/entrainer.py) et celle utilisee au moment de la prediction (ml_recommandation/classifier.py) : cette derniere fusionnait par erreur toutes les lettres "a" et "e" en "e" (residu d'une tentative de normalisation des accents devenue un artefact apres une manipulation anterieure du fichier). Ce decalage degradait silencieusement la qualite des predictions en production sans qu'aucune erreur ne soit levee. Les deux fonctions ont ete unifiees.

4.5 Resultat final retenu

Indicateur	Valeur
Exemples d'entrainement	1 201 (200 originaux + 840 augmentes + 361 reels plafonnes)
F1-score pondere (split 80/20)	0.922
F1-score - validation croisee 5-fold	0.919 (+/- 0.008)
Score sur 20 questions realistes independantes	18/20 (90%)

Les 2 erreurs restantes sur le jeu de test independant concernent des cas ambigus meme pour un humain : un litige de remboursement en magasin (attendu droit_commercial, predit droit_immobilier) et une convocation au commissariat sans motif precise (attendu droit_penal, predit droit_travail). Ces limites sont documentees plutot que masquees.

5. Synthese

Composant	Donnees utilisees	Methode	Etat
Chatbot (RAG)	9 392 lignes (article + query_variant)	Recherche par similarite TF-IDF, citation de la reference exacte	Integre, reentraine et valide
Recommandation	1 201 exemples (dont 361 issus des lots reels, plafonnes)	Classifieur TF-IDF + LinearSVC reentraine (class_weight='balanced')	Integre, reentraine et valide

Pistes d'amelioration identifiees pour la suite : lots supplementaires sur le bail locatif moderne, la creation d'entreprise (demarches CEPICI) et la fiscalite (CGI/TVA) afin de combler les zones faibles du corpus actuel ; possibilite d'ajouter une etape de vulgarisation automatique (LLM) du texte de loi brut cite par le chatbot pour les utilisateurs non-juristes.